

PCU-Select: Amortizing Target-Specific Data Selection Across PEFT Configurations

Anonymous submission

Abstract

A large language model (LLM) serving system typically maintains a registry of fine tuning configurations to serve diverse tasks and deployment budgets. These configurations vary in which modules are trained and how they are updated, leading to different trainable subspaces. This diversity poses a challenge for gradient based data selection: it must rebuild a target specific gradient datastore for every configuration, making cost scale linearly with registry size. Reusing a single subset across all configurations avoids this cost, but degrades quality because example utility is tightly coupled to the trainable subspace. We present PCU Select, which amortizes data selection across a configuration registry by conditioning utility on the task and a structured code that encodes the fine tuning strategy. The key insight is to decouple gradient recomputation from the target configuration through 24 shared intervention sites across the Transformer backbone. These sites provide a common basis for comparing different fine tuning methods. PCU Select then extracts reusable representations offline, generates multi fidelity utility labels using cheap gradient approximations calibrated by short fine tuning runs, and trains a conditional scorer to jointly predict utility and uncertainty. At deployment, the scorer ranks cached features for any target configuration and task without rebuilding gradients. On Llama 2 7B across four tasks and five configurations, PCU Select reduces the cost of onboarding a new configuration by 44 times compared to per configuration LESS, from 63.2 to 1.44 GPU hours, while maintaining downstream quality within a negligible margin of the baseline. By amortizing per configuration computation into a one time offline stage, PCU Select makes gradient level data selection practical at registry scale.

1 Introduction

Large language model platforms must adapt one backbone to different tasks and resource limits. Parameter-efficient fine-tuning (PEFT) reduces adaptation cost by updating only a small set of model parameters, allowing a platform to maintain a *registry* of configurations that differ in method, trainable modules, capacity, or training recipe. These trainable parameters determine a configuration’s *trainable subspace*: the

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

update directions available during fine-tuning. Each training example can affect the model only through the directions that the target configuration exposes. Data selection reduces cost along a complementary axis by choosing a compact subset whose examples produce useful task updates. Its decisions therefore depend on the target trainable subspace: changing the subspace can change which examples the selector should prioritize.

Most data selectors ignore this interaction. Quality filters, diversity heuristics, and proxy-model scores produce one ranking for a task and reuse it across PEFT configurations. We call such a ranking *PEFT-agnostic* because the selector does not receive the target configuration. Yet common PEFT methods permit different updates: LoRA adds low-rank changes to selected projections (Hu et al. 2022); adapters insert trainable bottlenecks (Houlsby et al. 2019); and (IA)³ scales selected activations (Liu et al. 2022). The same example can therefore produce different updates, and hence have different value, under different configurations.

Gradient-based selectors capture this dependence but incur repeated computation. LESS (Xia et al. 2024) first builds a target-specific *gradient datastore*, a cache containing a training gradient for each candidate example. It then ranks the candidates against a *task sketch*, a small set of training or development examples that represents the target task. Because each PEFT configuration exposes different trainable parameters, LESS must rebuild the cache for every configuration. Its selection cost therefore grows linearly with the registry.

One shortcut is to reuse a single LESS subset for every configuration. This reuse avoids repeated computation but removes the configuration information that makes gradient selection useful. In our evaluation, the shortcut performs worse than a PEFT-agnostic representation baseline (Table 2). Existing methods thus force a choice: recompute a target-specific ranking for every configuration or reuse a ranking that ignores the configuration.

The configuration dependence affects downstream performance as well as the rankings. Figure 1 shows that independent runs for the same configuration agree more closely than runs across configurations. Changes within one PEFT family preserve more of the ranking, while cross-family changes produce the largest disagreement. When selection and training use the same PEFT, the resulting subsets outperform

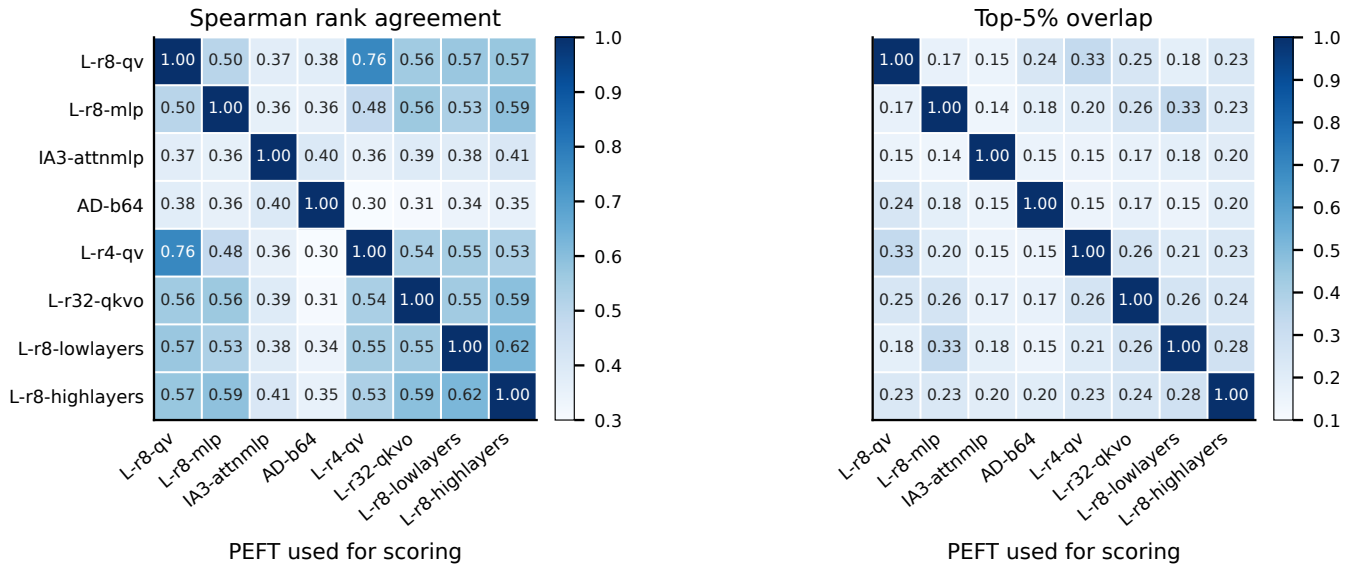


Figure 1: Short-update estimates—task-sketch loss reductions after a few PEFT updates—yield different data rankings across configurations. We score a common pool of 20K candidate examples using two task sketches and independent checkpoint/seed replicates. Each cell reports the Spearman rank correlation between its row and column runs; the diagonal compares each run with itself. Independent runs for the same PEFT reach $\rho = 0.79$ and Top-5% overlap = 0.32, providing a reference for variation in these labels. Cross-family pairs fall to $\rho = 0.36$ and Top-5% overlap = 0.17 on average.

cross-family mismatched selection by 2.0 points on average (Appendix Table 12). The same pattern appears within LoRA configurations, where matched selection improves performance by 1.45 points (§6).

These observations motivate *PEFT-conditioned utility*: the value of training on example x for task t under PEFT configuration p . We denote this value by $u(x, p, t)$. A task-only selector omits p and must assign the same ranking to every configuration. Per-PEFT influence selection includes p but recomputes gradients for each target. Our goal is to predict $u(x, p, t)$ while sharing the expensive computations used to learn these scores.

PCU-Select (PEFT-Conditional Utility) enables this sharing through a common representation. It maps every supported configuration to 24 *intervention sites*: selected Transformer module outputs through which PEFT updates affect the backbone. These outputs remain comparable even when PEFT methods train parameter tensors with different shapes. PCU-Select caches sample and task signals at the sites. A structured PEFT code records which sites a configuration affects, its update type, its capacity, and its training recipe. Supplying this code at scoring time preserves configuration information while keeping the cached signals reusable.

A conditional scorer turns the shared signals into a target-specific ranking. Its prediction depends on the candidate example, task sketch, and PEFT code. It learns from two complementary signals: inexpensive labels based on gradient alignment cover the full candidate pool, while short-update labels measure task-sketch loss reductions for a smaller sample. At deployment, the scorer ranks the cached examples for a target configuration and task. PCU-Select then spreads the budget across groups of semantically similar examples

to limit near-duplicates. This pipeline moves gradient extraction and label construction offline while retaining a distinct subset for each target.

These design choices amortize target-specific selection across a PEFT registry. We make three contributions:

- **PEFT-conditioned utility.** We formulate data value as $u(x, p, t)$ and show that both data-value rankings and downstream subset quality depend on the PEFT configuration (Figure 1; Appendix Table 12).
- **Intervention-site amortization.** PCU-Select represents different PEFT parameterizations through the same 24 module-output sites. A structured code preserves each target’s update locations, type, capacity, and recipe without rebuilding a gradient datastore. Removing the PEFT code or task sketch reduces downstream performance by 1.37 or 0.82 points, respectively (Appendix Table 11).
- **Registry-scale quality and cost.** Across four tasks and five seen PEFTs on Llama-2-7B (Touvron et al. 2023), PCU-Select gains 0.74 points over the PEFT-agnostic representation baseline RDS+ and differs from per-PEFT LESS by +0.04. It reduces the selection cost of adding one four-task PEFT by $44 \times$ (1.44 versus 63.2 GPU-hours; Tables 1 and 2).

2 Related Work

Instruction-data selection. Instruction-data selectors usually estimate quality or diversity independently of the target PEFT configuration. LIMA supports instruction tuning with 1,000 curated examples (Zhou et al. 2023). Model-based filters instead use judgments or instruction properties: Alpaca filters Alpaca examples with LLM judgments (Chen

et al. 2024), InsTag measures semantic diversity and complexity through instruction tags (Lu et al. 2023), and DEITA combines quality, complexity, and diversity (Liu et al. 2024). Other methods select for explicit diversity through quality-diversity trade-offs, log-determinant objectives, or partition-aware sampling (Bukharin et al. 2023; Wang et al. 2024; Rao et al. 2024). These methods improve data efficiency, but their rankings omit the target PEFT’s trainable parameters.

Proxy-model and influence selection. Proxy-model selectors provide reusable rankings without computing gradients for every target PEFT. Cherry LLM uses instruction-following difficulty (Li et al. 2023); Superfiltering studies when weak models can rank data for stronger models (Li et al. 2024); and SmallToLarge transfers training-trajectory information from small models to larger targets (Yang et al. 2024). Because these scores encode only example and task information, a PEFT change leaves the ranking unchanged.

Influence-based selectors model training effects more directly, but their gradients tie the score to a target parameterization. Classical influence functions use second-order approximations (Koh and Liang 2017), while TracIn accumulates gradient similarities across checkpoints (Pruthi et al. 2020). LESS builds a low-dimensional gradient datastore for instruction selection (Xia et al. 2024), and DataInf specializes influence approximation for LoRA-tuned generative models (Kwon et al. 2024). Preserving configuration specificity therefore requires rebuilding PEFT-dependent gradients when the target changes. PCU-Select instead conditions one scorer on a structured PEFT code, combining reusable sample signals with a target-specific ranking. Our evaluation compares PCU-Select with both a per-PEFT LESS implementation and an Influence baseline that uses one shared datastore.

Parameter-efficient fine-tuning. PEFT methods reduce adaptation cost in different ways, so they expose different trainable parameters. Adapters insert bottleneck modules into Transformer layers (Houlsby et al. 2019; Pfeiffer et al. 2021), while prefix and prompt tuning optimize continuous prefix states or prompts (Li and Liang 2021; Lester, Al-Rfou, and Constant 2021). LoRA learns low-rank additive updates (Hu et al. 2022), and QLoRA combines quantization with LoRA to reduce memory use (Dettmers et al. 2023). BitFit tunes bias terms (Ben Zaken, Goldberg, and Ravfogel 2022), whereas (IA)³ rescales attention and feed-forward activations (Liu et al. 2022). Prior PEFT studies usually compare accuracy, parameter count, memory, or inference overhead while holding the training data fixed. We study the complementary question: whether a fixed candidate pool has the same example ranking across PEFT configurations.

Joint data- and parameter-efficient adaptation. The closest methods use PEFT gradients for data selection, but their utility estimates remain specific to one target configuration. DataInf and LESS compute selection signals from LoRA-style training (Kwon et al. 2024; Xia et al. 2024). Geometry-aware targeted selection further shows that useful data depends on the directions available to PEFT optimization (Min et al. 2026). These methods must regenerate

configuration-dependent signals when the target changes.

PCU-Select instead describes each configuration by its intervention sites, update type, capacity, and training recipe. This representation lets one scorer reuse sample and task signals while still producing a different ranking for each configuration. The next section formalizes this requirement and the resulting cost objective.

3 Problem Formulation

We study *cross-PEFT data-efficient fine-tuning*: selecting a small training subset for a target task and PEFT configuration while reusing selection work across configurations. We fix a backbone family and a frozen base model M_0 . The candidate pool $\mathcal{D} = \{x_i\}_{i=1}^N$ contains N instruction–response examples. Each task t provides a *task sketch* V_t , a small set from its training or development data. Each PEFT configuration $p \in \mathcal{P}$ specifies a method, trainable locations, capacity, and training recipe. For a selection budget B , the ideal subset maximizes held-out task performance:

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{D}, |\mathcal{S}| \leq B} \text{Perf}(\text{Tune}(M_0, p, \mathcal{S}), V_t^{\text{test}}). \quad (1)$$

Here Tune adapts M_0 with configuration p and subset $\mathcal{S}$, and Perf evaluates the adapted model. The held-out test set V_t^{test} defines this ideal objective but never enters training, scoring, or selection; those procedures use only V_t . Directly solving Eq. (1) would require training and evaluating many subsets for every target (p, t) pair.

PCU-Select avoids this search by assigning each example a conditional utility: its benefit for task t when training uses configuration p . The scorer s_ϕ approximates this utility for each example:

$$s_\phi(x, p, t) \approx u(x, p, t). \quad (2)$$

The target $u(x, p, t) \in [0, 1]$ summarizes task-sketch loss reductions after a few PEFT update steps. PCU-Select rank-normalizes these reductions within groups that share the same PEFT, task, and measurement setting (§4). Including p allows the ranking to change with the trainable subspace; omitting p would force every configuration to share one task-specific ranking. The task sketch V_t identifies the desired task behavior, while p identifies the updates available to the model. Utility is therefore relative to both the task and the PEFT configuration.

Amortization objective. A registry changes the cost objective because it serves multiple PEFT configurations and tasks from one candidate pool. Let P be the number of target configurations and Q the number of task sketches. LESS builds one gradient datastore for each configuration and ranks that datastore once for each task. PCU-Select instead pays one shared offline cost, followed by one scoring-and-selection pass for every PEFT–task pair:

$$\begin{aligned} C_{\text{LESS}}(P, Q) &= P(C_{\text{LESS}}^{\text{data}} + QC_{\text{LESS}}^{\text{rank}}), \\ C_{\text{PCU}}(P, Q) &= C_{\text{offline}} + PQ C_{\text{PCU}}^{\text{pair}}. \end{aligned} \quad (3)$$

Equation (3) separates four costs. $C_{\text{LESS}}^{\text{data}}$ builds one PEFT-specific datastore, and $C_{\text{LESS}}^{\text{rank}}$ ranks it for one task. C_{offline}

extracts reusable signals and trains PCU-Select once. $C_{\text{PCU}}^{\text{pair}}$ scores and selects data for one target pair. The first expression repeats datastore construction P times, whereas the second pays the offline cost once. Section 5 measures each cost term.

Lower cost matters only if the selected subset preserves target-specific quality. We express the quality difference from per-PEFT LESS as

$$\Delta_{\text{quality}} = \text{Perf}_{\text{PCU}} - \text{Perf}_{\text{LESS}}, \quad |\Delta_{\text{quality}}| \leq \epsilon, \quad (4)$$

where Perf_{PCU} and $\text{Perf}_{\text{LESS}}$ denote their downstream scores and ϵ is the acceptable difference. The cost equations also identify when the shared offline investment pays off. For a fixed Q , PCU-Select costs less once P exceeds $P^* = C_{\text{offline}} / [C_{\text{LESS}}^{\text{data}} + Q(C_{\text{LESS}}^{\text{rank}} - C_{\text{PCU}}^{\text{pair}})]$, assuming the denominator is positive. The objective is therefore to retain per-PEFT LESS quality while replacing repeated datastore construction with shared offline work.

Design requirements. The formulation yields four design requirements. **PEFT conditioning** includes p so the example ranking can change with the target configuration. **Task conditioning** uses V_t so the model learns task-specific value instead of generic example quality. **Structured PEFT code** records where and how each PEFT updates the model in a form shared across families. **Reusable execution** moves gradient extraction, label construction, and scorer training offline, leaving cached-feature scoring for each new (p, t) pair. The next section shows how PCU-Select meets these requirements.

4 Method: PCU-Select

PCU-Select learns the conditional utility surrogate in Eq. (2) and selects a budgeted subset for a target PEFT-task pair. Figure 2 summarizes two stages. Offline, it extracts reusable sample and task signals, constructs two types of utility label, and trains one scorer. Online, it combines cached sample features with a target (p^*, t^*) , scores the pool, and distributes budget across semantic clusters. Algorithms 1 and 2 specify both stages.

Shared Intervention Sites Enable Cross-PEFT Reuse

PEFT families expose parameter tensors with different shapes and roles, so their raw gradients cannot share one cache (Hu et al. 2022; Houlsby et al. 2019; Liu et al. 2022). PCU-Select instead uses a fixed set Ω of *intervention sites*. Each site is a selected module output through which a PEFT update affects the backbone. The set Ω is a shared coordinate system because sample gradients, task gradients, and each PEFT’s map of affected sites all use the same index ω .

PCU-Select hooks attention, MLP, and post-residual outputs at eight evenly spaced Transformer layers, giving $|\Omega| = 24$. A site’s response-loss gradient describes how a sample or task pushes that activation. By the chain rule, PEFT-parameter gradients factor through gradients at the module outputs they modify. Site gradients therefore avoid both full PEFT-parameter reconstruction and shape mismatches between fused and split projections. Post-residual sites place adapters in the same index space as LoRA and (IA)³.

A site index records only *where* a configuration acts. PCU-Select separately encodes operator and capacity features to record *how* it acts and how much it can update. This factorization reuses one sample signature without treating additive, multiplicative, and bottleneck updates as equivalent.

Each configuration maps to a site mask and operator type. Attention LoRA and (IA)³-attention map to attention-output sites. MLP LoRA and (IA)³-FFN map to MLP-output sites, while adapters map to block-residual sites. Prefix tuning maps to attention-output sites; BitFit maps to a bias shift. These mappings define each normalized site weight:

$$\begin{aligned} w_p^\omega &= \text{mask}(p, \omega) \kappa(p, \omega), \\ \kappa(p, \omega) &= \tanh(\eta \log(1 + \text{cap}(p, \omega)/d_{\text{model}})), \\ \tilde{w}_p^\omega &= w_p^\omega / \max(\varepsilon_w, \sum_{\omega'} w_p^{\omega'}). \end{aligned} \quad (5)$$

Here $\text{mask}(p, \omega)$ is zero at untouched sites, and $\text{cap}(p, \omega)$ measures trainable capacity. The width d_{model} normalizes capacity, η controls saturation through $\tanh$, and ε_w keeps the denominator nonzero. Thus, $\tilde{w}_p^\omega$ distributes unit weight over the sites used by p . Appendix Table 10 maps LoRA rank, adapter width, (IA)³ vector count, prefix length, and BitFit bias count to this scale. Operator type remains separate in z_p because a family-wide scalar would cancel under Eq. (5)’s normalization.

Shared Representations Preserve Target Specificity

Shared site indices make signals comparable, but the scorer must still distinguish each sample, task, and PEFT. PCU-Select uses three fixed-length vectors: the sample representation describes the example, the task representation describes the target need, and the PEFT representation describes where and how training acts. Site gradients supply separate label-construction signals.

The sample representation $z_x = [e_x; d_x; a_x] \in \mathbb{R}^{848}$ combines semantic, descriptive, and activation features. The 768-dimensional e_x is the frozen sentence-transformers/all-mpnet-base-v2 embedding of the instruction and response (Song et al. 2020; Reimers and Gurevych 2019). This encoder remains frozen during pool and target-benchmark processing. The 16-dimensional d_x records length, response-LM loss, entropy, and format. The 64-dimensional a_x summarizes response-token hidden states at the eight sampled layers.

Utility is task-relative, so PCU-Select forms $z_t \in \mathbb{R}^{848}$ by mean-pooling z_x over the task sketch’s 32 examples. For proxy labels, it separately averages their site gradients into g_t^ω . These 24×256 values remain outside z_t and serve only label construction.

The PEFT representation records the target trainable subspace. In $z_p = [m_p; c_p; r_p] \in \mathbb{R}^{128}$, the 24×4 mask m_p contributes 96 dimensions. Its channels mark active sites, additive updates, multiplicative scaling, and prefix-style control. Two 16-dimensional vectors encode capacity c_p and training recipe r_p . At this site resolution, q/v and q/k/v/o LoRA share attention-output sites, while capacity and recipe preserve their projection-level distinction.

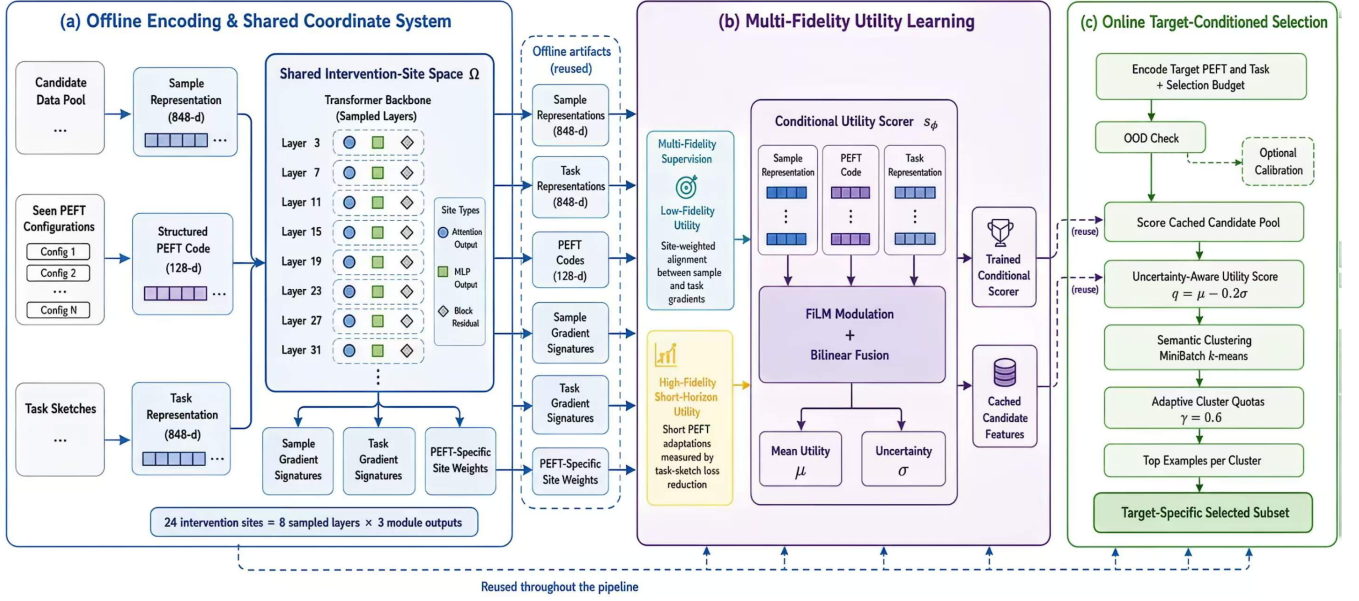


Figure 2: PCU-Select separates reusable offline supervision from online PEFT–task scoring. Its shared coordinate system Ω contains 24 intervention sites, or selected module outputs. Sample and task-sketch gradients use these site indices, while the PEFT code identifies the sites affected by each configuration. The 128-dimensional PEFT code comprises a 96-dimensional site/operator mask, 16 capacity dimensions, and 16 recipe dimensions. Low-fidelity gradient proxies provide broad coverage; high-fidelity short updates provide direct, costlier supervision.

Prefix, P-Tuning, and BitFit use the same deterministic encoding rules as seen families. The scorer has no learned family lookup or operator centroid. The support policy below decides whether a target uses direct scoring, calibrated scoring, or a per-target selector.

Training these representations requires labels for many (x, p, t) triples, but measuring every PEFT update would defeat amortization. PCU-Select instead uses *multi-fidelity supervision*, which combines labels with different costs and directness. A low-fidelity gradient proxy cheaply covers the pool. High-fidelity short updates directly measure sketch-loss reduction for fewer triples. Together, they learn broad ranking structure and correct errors near the subset cutoff.

The *gradient-proxy label* measures alignment between a sample and the task sketch at sites used by the target PEFT:

$$u^{\text{lo}}(x, p, t) = \sum_{\omega \in \Omega} \tilde{w}_p^\omega \cos(g_x^\omega, g_t^\omega). \quad (6)$$

Here g_x^ω is the response-token loss gradient at site ω . A Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) compresses it to 256 dimensions and unit-normalizes the result. The task-sketch mean gives g_t^ω . Equation (6) therefore measures alignment only at sites used by p . All sketches use response-token cross-entropy for these signatures; task-native metrics such as option likelihood, Pass@1, and F1 remain downstream evaluation metrics. Because cosine values can be negative, PCU-Select rank-normalizes them within each (p, t) bucket before proxy distillation and ranking. Cached sample signatures then label many (x, p, t) triples without new gradient extraction.

The *short-update label* measures realized sketch-loss reduction near the subset cutoff. For each sampled triple, PCU-Select uses two anchors θ_a : the base model and a warm checkpoint after adaptation has begun. The horizon h is the number of short-update steps. A mini-batch $\mathcal{B}(x)$ contains the sampled example and seven condition-matched fillers. PCU-Select applies PEFT p for h steps and measures

$$\Delta(x, p, t, a, h) = L_{V_t}(\theta_a) - L_{V_t}(\text{Adapt}_p^h(\mathcal{B}(x); \theta_a)). \quad (7)$$

Thus, Δ is the task-sketch loss before the update minus its loss afterward. Rank-normalized reductions for $h \in \{1, 4\}$ from both anchors define u^{hi} . The fillers make this mini-batch effect more stable than a single-example update. PCU-Select allocates 10K short-update labels in three stages. Coverage sampling first spreads labels across the registry without a scorer.

Coverage labels and the proxy then train a provisional scorer. It predicts ranks and a per-triple uncertainty scale, an estimate of prediction noise. Uncertainty sampling selects triples with large scales. Boundary sampling selects large predicted–observed rank disagreements near the cutoff. Every rank normalization compares labels within one PEFT, task, anchor, and horizon bucket.

The proxy cheaply orders the full pool, while short updates target consequential ranking errors. Two anchors test whether utility persists after adaptation begins; two horizons reduce dependence on one update length. Ranking within condition-matched buckets avoids assuming a shared raw-loss scale across tasks or configurations.

Conditional Scoring Produces Target-Specific Subsets

The scorer consumes (z_x, z_p, z_t) and predicts $(\hat{\mu}, \hat{\sigma}) = s_\phi(z_x, z_p, z_t)$. Here $\hat{\mu}$ gives predicted mean utility, while $\hat{\sigma}$ gives a positive, input-dependent uncertainty scale. Separate encoders map the three inputs into hidden vectors. FiLM modulation (Perez et al. 2018) uses the PEFT and task codes to rescale and shift sample features. A bilinear term represents sample-PEFT compatibility.

Stage A distills gradient-proxy ranks to learn broad conditional structure. Stage B continues proxy distillation and adds ranking and regression on short-update labels plus a heteroscedastic uncertainty loss. Here *heteroscedastic* means that predicted noise can differ across triples. PCU-Select selects the Stage B checkpoint on held-out PEFT-task pairs. It also fixes the sketch size, label allocation, anchors, horizons, and scalar selection hyperparameters on these pairs before downstream evaluation.

A target’s structural support level determines whether PCU-Select applies the scorer directly or calibrates it. Relative to the seen registry, L0 resembles a seen configuration, L1 makes a larger shift within a seen family, and L2 introduces an unseen family. Seen targets use the trained scorer directly. L0 targets run *zero-shot*: PCU-Select scores a new configuration without target-specific short-update labels. L1 and L2 targets use calibrated scoring when compatible labels exist and a per-target selector otherwise.

After any required calibration, PCU-Select converts the two scorer outputs into a conservative ranking score:

$$q(x) = \hat{\mu}(x, p^*, t^*) - \lambda \hat{\sigma}(x, p^*, t^*), \quad \lambda = 0.2. \quad (8)$$

The uncertainty scale reduces the rank of examples whose predicted utility is noisy. It serves as a ranking penalty, not a confidence interval.

Ranking by q alone can over-select near-duplicates, so PCU-Select clusters examples in semantic space before assigning the budget. It allocates budget B across K clusters C_k , where $k \in \{1, \dots, K\}$:

$$b_k = \text{round} \left(B \frac{(v_k^+)^{\gamma} |C_k|^{1-\gamma}}{\sum_j (v_j^+)^{\gamma} |C_j|^{1-\gamma}} \right), \quad \gamma = 0.6. \quad (9)$$

Here v_k is the mean conservative score among the top 10% of examples in C_k , and $v_k^+ = \max(v_k, 0)$. The factors v_k^+ and $|C_k|$ balance cluster value and size. For the 300K pool, $K = \max(50, \lfloor \sqrt{N} \rfloor)$ produces 547 clusters of about 548 examples. The reported 0.36 GPU-hours per PEFT-task application includes clustering. PCU-Select takes the top- b_k examples by q within each cluster. Non-positive clusters receive no budget; deterministic remainder handling makes the quotas sum to B .

For seen and L0 targets, all gradient extraction and label construction occur offline; online selection uses cached features and scorer forward passes. L1 and calibratable L2 targets add a small label set without rebuilding a target-specific gradient datastore. The evaluation next tests whether this reuse preserves LESS-level quality, reduces registry cost, and transfers across support levels.

5 Experiments

The evaluation tests three claims: PCU-Select matches per-PEFT LESS on downstream quality, reduces the marginal cost of adding a PEFT, and transfers to new configurations according to their structural support in the training registry.

Setup

We evaluate on GSM8K, HumanEval, MMLU, and TyDiQA (Cobbe et al. 2021; Chen et al. 2021; Hendrycks et al. 2021; Clark et al. 2020), using exact match, Pass@1, accuracy, and F1, respectively. We express these scores as percentages before averaging. From a pool of 300K mixed instruction examples, every selector chooses 10%.

The main comparison covers five configurations seen during scorer training: attention and MLP LoRA, (IA)³, and adapters. The transfer study holds out configurations that vary rank, placement, learning rate, or family. Appendix Table 5 lists the full 17-configuration registry.

The four baselines differ in target specificity. **Random** uses neither task nor PEFT information. **RDS+** ranks frozen semantic embeddings by similarity to the task-sketch mean, but ignores the PEFT configuration. **Influence** ranks projected base-model gradients from one shared, PEFT-agnostic datastore. **LESS**, our per-PEFT implementation of gradient-influence selection (Xia et al. 2024), warms up each target adapter, builds an Adam-preconditioned gradient datastore, and ranks examples by sketch influence. PCU-Select uses its conditional scorer, uncertainty penalty, and adaptive semantic-cluster quotas. Appendix A gives full specifications.

We hold the backbone, PEFT recipe, training steps, and evaluation protocol constant, so only the selected subset changes. The main table varies target-training seeds 0, 1, and 2 while fixing the selector, scorer, sketch, and pool at seed 0. Matched seeds give every selector the same training randomness. We obtain each descriptive 95% interval by resampling the evaluated PEFT-task cells under this fixed selection pipeline.

PCU-Select Matches Per-PEFT LESS

Table 1 shows that PCU-Select improves on the PEFT-agnostic baselines and remains close to target-specific LESS. It gains 0.74 points over RDS+ and wins 17 of 20 PEFT×task cells, with descriptive interval $[+0.38, +1.06]$. It gains 0.50 points over Influence and wins 18 cells, with interval $[+0.21, +0.75]$. Its comparison with LESS is closer: a 0.04-point gap, interval $[-0.26, +0.33]$, and 10 wins.

The LESS comparison varies across tasks. PCU-Select gains on MMLU, stays within seed dispersion on GSM8K and HumanEval, and trails on TyDiQA and MLP-only LoRA. Task-grouped resampling yields $[-0.15, +0.22]$ for the aggregate gap (Appendix Table 3). **Takeaway.** Shared PEFT-conditioned supervision improves on PEFT-agnostic selection while preserving per-PEFT LESS’s average quality on the evaluated registry.

Amortization Cuts Marginal Cost by 44×

Table 2 separates *fixed/shared* cost, paid once, from *marginal* cost, paid for each new PEFT. All selector costs exclude the